# voipred.dev: tests and messages

Net benefit, value of information and plots for binary, survival and competing-risks outcomes

17 October 2026


- Summary
- Part 1: tests
  - Tests by area
  - Models checked
- Part 2: error and warning messages
  - Arguments of nb() (12)
  - Outcome (11)
  - Survival arguments (11)
  - Predicted risks given directly (6)
  - Model and covariates (9)
  - Predicted risks (8)
  - Specific models (20)
  - The prior (4)
  - cIPCW: cipcw_control() (11)
  - cIPCW: censoring model (9)
  - cIPCW: censoring weights (10)
  - Value of information (5)
  - cIPCW: refit in each draw (4)
  - print() and summary() (2)
  - Plots (18)
- Appendix: every test


# Summary


The full test suite of voipred.dev (on top of commit c306747) was run on 17 October 2026 with R version 4.3.3 (2024-02-29): **201 tests with 1,518 checks (expectations), 0 failures, 0 skipped, 0 warnings.** It takes about 12 minutes.

The tests check voipred against independent references:

- **dcurves**: for every model, the net benefit equals `dcurves::dca()` on the same predicted risks at every threshold from 0 to 0.99 (tolerance $10^{-10}$); Kaplan-Meier for survival and Aalen-Johansen for competing risks.
- **The models' own predictions**: the risk voipred gets from each model equals the model package's own prediction, or an independent calculation for survival models (tolerance $10^{-6}$).
- **cIPCW**: the dcurves cIPCW branch (a saved reference), a direct implementation of the estimator, and the true net benefit in simulations with informative censoring.
- **predtools** ( `evpi_val()` , the code of Sadatsafavi et al. 2023): EVPI and P(useful) with the same Bayesian bootstrap draws (tolerance $10^{-12}$).
- **The EVSI paper's code** (Sadatsafavi et al. 2025, resplab/papercode and resplab/evsiexval): the Bayesian bootstrap EVSI and the conjugate `EVSI()` , with the same random numbers.

- **bayesDCA**: the posterior draws of the net benefit equal bayesDCA's with the same seed.
- **The usual net benefit**: with the default prior, `nb_improper()` (Beta(0, 0), the data only), the net benefit equals TP/n - FP/n z/(1-z) exactly, and its posterior draws equal the Bayesian bootstrap in distribution (means, spread, correlation between models, EVPI, P(best) and EVSI).
- **Hand calculations** and direct loops for the core estimator; the compiled Kaplan-Meier code equals the R implementation.

Part 1 summarizes the tests by area and lists the models checked, with every test listed at the end of the document. Part 2 lists every error and warning message voipred can give.

# Part 1: tests

## Tests by area

| Area | File | What is checked | Reference | Tests | Checks |
|---|---|---|---|---|---|
| Net benefit, binary | test-nb.R | Hand calculation; treated if risk >= threshold, risk 0 never treated; thresholds kept in order; vectorized estimator equals a direct loop; outcome as column, vector, logical or factor; integer weights equal duplicated rows; risks given directly; the result object; data first (old argument order refused); input errors; print. | Hand calculation; direct loop | 13 | 45 |
| Predicted risks given directly | test-risk-input.R | predicted_risk (vector or column) gives the same results as the model for binary, survival and competing risks, KM and cIPCW, other priors, and the value of information; labels; risks predicted once and reused by every later step; input errors. | The same model's results | 4 | 27 |
| Agreement with dcurves | test-dcurves.R | Logistic regression, ranger (with tied risks), randomForest and bayespred: model, treat all and treat none at every threshold. | dcurves::dca() | 4 | 17 |
| Binary models | test-models.R | Every supported binary model class gives the same net benefit as dcurves; log-odds, raw scores, decision values, discriminant scores and predicted classes are rejected; matrix models need numeric feature columns. | dcurves::dca() | 26 | 114 |
| Probabilities, not scores or classes | test-probabilities.R | Only 0s and 1s are rejected as classes; exact 0s and 1s among probabilities are accepted; values outside [0, 1] are rejected as scores. | Expected errors | 8 | 16 |
| Risks from a model (internal) | test-predict_risk.R | The risk equals the package's prediction (glm, ranger, randomForest, bayespred); multi-class forests, vectors and objects without predict() are rejected. | Package predictions | 12 | 15 |
| How risks are obtained | test-prediction-routes.R | predict_function takes precedence; it gets horizon and cause and is checked like any model; output shapes (vector, class columns, survival curve); order of the standard routes; a new model works with predict_function; the prediction code is internal (not exported). | Expected values and errors | 8 | 37 |
| Missing covariates | test-covariates.R | Missing covariates are named (glm, formula y ~ ., transformed covariates, randomForest, bayespred); ranger's own error is shown; extra columns are fine. | Expected errors | 8 | 14 |

| Area | File | What is checked | Reference | Tests | Checks |
|---|---|---|---|---|---|
| Survival outcome and arguments | test-survival-outcome.R | Outcome as formula, Surv object or column; horizon, cause, censoring and cipcw arguments are checked; only right censoring; missing or negative times. | Expected values and errors | 11 | 43 |
| Kaplan-Meier / Aalen-Johansen | test-survival-km.R | Net benefit equals dcurves at every threshold, including NA thresholds; internal KM and Aalen-Johansen equal survival::survfit (with weights); weights equal duplicated rows; KM biased under informative censoring. | dcurves::dca(); survival::survfit; simulation truth | 10 | 100 |
| Compiled Kaplan-Meier code | test-km-compiled.R | The compiled KM / Aalen-Johansen net benefit equals the R implementation (tolerance 10^-14), with random, zero and integer weights, ties and early horizons. | R implementation | 1 | 162 |
| Survival models | test-survival-models.R | Each survival model class: risk equals an independent calculation and the net benefit equals dcurves; models without probabilities are refused (also a model whose formula is in a variable); binary models with a survival outcome warn; horizon beyond follow-up warns; risk grows with the horizon. | Independent risk; dcurves::dca() | 19 | 215 |
| Competing risks | test-survival-competing.R | CSC, FGR, crr, competing-risks random forest and multi-state coxph: risk equals the package's cumulative incidence and the net benefit equals dcurves (Aalen-Johansen); causes by label or position; FGR fitted for another cause is refused. | Package CIF; dcurves::dca() | 10 | 97 |
| Conditional IPCW | test-cipcw.R | Equal to the dcurves cIPCW branch; equal to a direct implementation; a model gives the same as its weights by hand; close to the truth under informative censoring (also competing risks); censoring model checks; truncation and positivity; ties; case weights; every censoring model class; censoring predict_function. | dcurves cIPCW branch; direct code; simulation truth | 20 | 129 |
| The prior of nb() | test-bayes-nb.R | Posterior means equal the Beta model by hand for the uniform, Jeffreys and custom priors; the default improper prior (the data only) equals the usual net benefit exactly (also with case weights); counts of 0 handled as the usual estimate; prior constructors and printing; plots refuse different priors; weights only with the improper prior; survival only with the improper prior; input errors. | Hand calculation | 4 | 46 |

| Area | File | What is checked | Reference | Tests | Checks |
|---|---|---|---|---|---|
| Value of information | test-voi.R | Draws with weights 1 reproduce nb() for every outcome and estimator; EVPI and P(useful) equal predtools::evpi_val(); Bayesian bootstrap EVSI equals the paper's algorithm; P(useful), P(best), relative EVPI by their definitions; survival without censoring gives the binary results; case weights; NA thresholds; unknown arguments (no `method`); p_useful() takes one model; input errors. | predtools; EVSI paper code; definitions | 11 | 87 |
| cIPCW refit in each draw | test-voi-refit.R | Every known censoring model (coxph, survreg, cph, psm, flexsurvreg, prodlim) is refitted with the draw's weights (equal to duplicated rows); other models need refit_function (equal to duplicated rows); shared refits; EVPI/EVSI with refits, also competing risks; given weights refused. | Duplicated rows; direct refits | 7 | 49 |
| Value of information: posterior draws | test-bayes.R | One model: posterior draws and P(useful) equal bayesDCA; EVSI equals the conjugate EVSI() of the paper; several models are drawn jointly, each keeping its own posterior; multinomial draws; different priors not compared; with the improper prior the posterior draws equal the Bayesian bootstrap in distribution. | bayesDCA; EVSI paper code; Bayesian bootstrap | 6 | 77 |
| Result objects: print(), summary(), as.data.frame() | test-methods.R | Each result is a list (not attributes) with its parts; print() gives a header and the first rows; summary() gives the best strategy, sensitivity and specificity (share treated for survival), EVPI and the best strategy now, EVSI as % of EVPI, P(useful) bands and the most likely best strategy, checked by hand; chosen thresholds; as.data.frame() is the table. | Hand calculation; the results | 5 | 103 |
| plot() of nb() and delta_NB | test-plot.R | delta_NB = NB_model - max(NB_all, NB_none); labels; plot data equal the results; several results together; colours; incompatible results refused. | nb() results | 6 | 45 |
| VoI plots, interactive plots, DCA with EVPI/EVSI | test-plot-voi.R | plot() of evpi() (EVPI, relative EVPI), evsi() (chosen thresholds), p_useful() (one model per result, several results on one plot), p_best(): curves equal the results; plot() of nb() with EVPI/EVSI below (shared axis, a legend per panel); interactive = TRUE gives the same curves; shared look and legend placement; axis names; plotly-safe labels. | evpi(), evsi(), p_useful(), p_best() results | 8 | 80 |
| **Total** | | | | **201** | **1518** |

## Models checked

Each model is fitted on development data and evaluated on validation data (GUSTO-I for binary outcomes, Rotterdam for survival and competing risks). “Standard route” means voipred uses the model’s own `predict()` or `riskRegression::predictRisk()`; “voipred method” means voipred has its own (internal) prediction code for the model because the standard route fails or gives a wrong answer. This is only used when the model is given instead of the predicted risks.

| Outcome | Models | Route | Checked against |
|---|---|---|---|
| Binary | glm (logit, probit, cloglog), logistf, mgcv::gam, earth, rpart, partykit::ctree, randomForest, ranger (probability = TRUE), gbm, kknn, naivebayes, xgboost() | Standard route | dcurves::dca() at thresholds 0 to 0.99 |
| Binary | e1071::svm (probability = TRUE), e1071::naiveBayes, nnet, MASS::lda, MASS::qda, glmnet, cv.glmnet, xgb.train, lightgbm, catboost | voipred method | dcurves::dca() at thresholds 0 to 0.99 |
| Binary | bayespred bpmfit and bpm (posterior mean of risk) | Standard route | dcurves::dca(); bayespred’s own posterior mean |
| Survival | coxph with x = TRUE (Efron, Breslow, case weights, ties, strata, splines), rms::cph, rms::psm, survreg, flexsurvreg, flexsurvspline, ranger, randomForestSRC, prodlim | Standard route | Independent risk by the horizon; dcurves::dca() (KM) |
| Survival | coxph without x = TRUE, stratified survreg, aorsf, partykit::ctree, rstpm2::stpm2, mgcv::gam (cox.ph) | voipred method | Independent risk (e.g. survival::basehaz, survreg quantiles); dcurves::dca() (KM) |
| Competing risks | riskRegression::CSC, riskRegression::FGR, competing-risks randomForestSRC | Standard route | Package cumulative incidence; dcurves::dca() (Aalen-Johansen) |
| Competing risks | cmprsk::crr, survival’s multi-state coxph | voipred method | Package cumulative incidence; CSC; dcurves::dca() (Aalen-Johansen) |
| Censoring model (cIPCW) | coxph, survreg (also stratified), rms::psm, flexsurvreg, flexsurvspline, ranger, randomForestSRC, aorsf, partykit::ctree, rstpm2::stpm2, mgcv::gam, prodlim (Kaplan-Meier); any other model with predict_function | Standard route or voipred method | The model’s own censoring survival; same net benefit as its weights given by hand |

bayespred.dev Bayesian survival models (Cox and parametric) are also supported, as models and as censoring models, but bayespred.dev is a private package, so they are not part of the test suite.

The test suite also checks that models which only give scores, relative risks, times or classes are refused with a clear message (e.g. log-odds from `glm`, `glmnet` and `gbm`; XGBoost `binary:logitraw`; LightGBM raw scores; CatBoost raw values and classes; SVM decision values; LDA scores; `ranger` and `svm` without `probability = TRUE`; `xgboost` survival objectives, `gbm` and `glmnet` Cox models, `rpart` survival trees).

# Part 2: error and warning messages

voipred has 127 error messages and 13 warnings. They are listed below by what they are about, as written in the code. Text in braces, such as `{n}` or `{class}`, is filled in with the actual values (e.g. the number of rows or the class of the model).

## Arguments of nb() (12)

| Type | Message |
|---|---|
| Error | `data` must be a data frame. The first argument of nb() is `data`: nb(data, predicted_risk, outcome, ...). |
| Error | `data` must be a data frame. |
| Error | Give either `predicted_risk` or `model`, not both. |
| Error | `cipcw` must be made with cipcw_control(). |
| Error | `predict_function` predicts from `model`; it is not used with `predicted_risk`. |
| Error | Give the predicted risks with `predicted_risk` (or a fitted `model`). |
| Error | `predict_function` must be a function(model, newdata, horizon, cause). |
| Error | `thresholds` must be a non-empty numeric vector. |
| Error | `thresholds` must lie in [0, 1). |
| Error | `label` must be a single non-empty string. |
| Error | `label` cannot be "{label}": "Treat All" and "Treat None" are the default strategies. |
| Error | `weights` must be {n} non-negative numbers with a positive sum. |

## Outcome (11)

| Type | Message |
|---|---|
| Error | An `outcome` formula must be one-sided, e.g. ~ Surv(time, status). |
| Error | Could not evaluate the `outcome` formula in `data`: {original error} |
| Error | Column '{column}' is not in `data`. |
| Error | `outcome` has length {m} but `data` has {n} rows. |
| Error | `outcome` has missing values. |
| Error | A factor outcome must have exactly two levels. |
| Error | `outcome` must be binary (0/1, logical, or a two-level factor) or a survival outcome made with Surv(). |
| Error | Only right-censored survival outcomes are supported: Surv(time, status). This outcome is of type '{type}'. |
| Error | `outcome` has {m} rows but `data` has {n} rows. |
| Error | The survival outcome has missing times or statuses. (For competing risks, give the status as a factor whose first level is censoring, e.g. Surv(time, factor(status, 0:2)); Surv() turns other numeric codes into NA.) |
| Error | The survival outcome has negative times. |

## Survival arguments (11)

| Type | Message |
|---|---|
| Error | `horizon` is only used with a survival outcome made with Surv(). |

| Type | Message |
|---|---|
| Error | `cause` is only used with a competing-risks outcome. |
| Error | `censoring` and `cipcw` are only used with a survival outcome. |
| Error | A survival outcome needs `horizon` : the time at which the risk is predicted (in the same units as the survival times). |
| Error | `horizon` must be a single positive number. |
| Error | `horizon` ({horizon}) is beyond the follow-up of every patient in `data` (longest follow-up: {longest follow-up}). |
| Error | `cause` is only used with a competing-risks outcome, i.e. Surv(time, status) with a factor status. |
| Error | `cause` must be one of the event types of the outcome: {event types}. |
| Error | `cipcw` settings were given but `censoring = "km"` . Use `censoring = "cipcw"` to apply them, or remove `cipcw` . |
| Error | `censoring = "cipcw"` needs a censoring model or weights: cipcw = cipcw_control(model = ...) or cipcw_control(weight_event = ..., weight_horizon = ...). |
| Warning | No patient treated at thresholds {thresholds} is followed up to the horizon ({horizon}), so the risk among them cannot be estimated: NB_model is NA there. |

## Predicted risks given directly (6)

| Type | Message |
|---|---|
| Error | `predicted_risk` names a column (“{column}”) that is not in `data` . |
| Error | `predicted_risk` must be numeric: one predicted probability per row of `data` , or the name of the column that holds them. |
| Error | `predicted_risk` has {m} values but `data` has {n} rows. |
| Error | {k} of the risks in `predicted_risk` are NA. |
| Error | `predicted_risk` has values outside [0, 1] (from {min} to {max}), so these are not probabilities (e.g. log-odds must be turned into probabilities first). |
| Error | `predicted_risk` has only 0s and 1s. These look like predicted classes, not probabilities. |

## Model and covariates (9)

| Type | Message |
|---|---|
| Error | Could not get predicted risks from this {class} model ({reason}). If a covariate is missing from the data, add it. Otherwise give `predict_function = function(model, newdata, horizon, cause)` to nb(), returning the risk of the event by the horizon. |
| Error | `model` must be a fitted model. To use predicted risks, give them with `predicted_risk =` instead. |
| Error | `data` must be a data frame. |
| Error | The validation data is missing {k} covariates that the {model or censoring model} was developed with: {covariates}. |
| Error | This {class} model was fitted to a survival outcome: use it with a survival outcome made with Surv() and give `horizon` . |
| Warning | The model ({class}) was fitted to a binary outcome, not a survival outcome. Its predicted probabilities are taken as the risk of the event by the horizon ({horizon}), which can be misleading if it was not built for this horizon and outcome. |
| Error | The model ({class}) is a competing-risks model, but the outcome has a single event type. Give a competing-risks outcome: Surv(time, status) with a factor status. |

| Type | Message |
|---|---|
| Warning | The model ({class}) was fitted to a standard survival outcome: its risk is used as the risk of '{cause}', but it is not the cumulative incidence of that cause when there are competing events, and can overstate the risk. Use a competing-risks model. |
| Warning | `horizon` ({horizon}) is beyond the last time in the {class} model's own data ({last time}); its predictions there are extrapolated or carry the last estimate forward. |

## Predicted risks (8)

| Type | Message |
|---|---|
| Error | A survival curve was returned, but there is no `horizon`. |
| Error | The model predicts {k} outcome classes; the outcome must be binary. |
| Error | The model returned classes, not probabilities. Make it return probabilities (or give `predict_function`). |
| Error | Predicted risks must be numeric. |
| Error | The model returned {m} predictions for {n} rows. Check for missing values in the covariates. |
| Error | {k} predicted risks are NA. Check for missing values in the covariates. |
| Error | The model returned values outside [0, 1] (from {min} to {max}), so these are not probabilities. They may be scores from the model (e.g. log-odds, relative risks or predicted times) that must be turned into probabilities first. |
| Error | The model returned only 0s and 1s. These look like predicted classes (labels), not probabilities. Make the model return probabilities (e.g. fit ranger or svm with `probability = TRUE`). |

## Specific models (20)

| Model | Type | Message |
|---|---|---|
| svm | Error | This svm was fitted without `probability = TRUE`, so it only predicts classes. Refit it with `probability = TRUE` to get predicted risks. |
| glmnet | Error | This glmnet model has {k} values of lambda. Fit it with a single lambda, or use cv.glmnet(). |
| catboost | Error | The catboost package is needed to predict from this model. |
| glmnet, xgboost, lightgbm, catboost | Error | Could not read the feature names from this {class} model. Train it on a matrix with column names. |
| glmnet, xgboost, lightgbm, catboost | Error | This {class} model was trained on a numeric matrix, but these columns of the validation data are not numeric: {columns}. Code them the same way as in the development data (e.g. as dummy columns). |
| gam | Error | This gam model (Cox PH family) is a survival model: use it with a survival outcome made with Surv() and give `horizon`. |
| bayespred.dev | Error | This bpmfit model is a survival model: use it with a survival outcome made with Surv() and give `horizon`. |
| bayespred.dev | Error | This bpm object has no baseline hazard, so it gives no risks. Package the fit at the horizon: posterior(fit, time = {horizon}). |
| bayespred.dev | Error | This bpm object is a survival model packaged at time = {time}: use it with a survival outcome made with Surv() and horizon = {time}. |

| Model | Type | Message |
|---|---|---|
| bayespred.dev | Error | This bpm object gives risks at time = {time}, but `horizon` is {horizon}. Use horizon = {time}, or package the fit at the horizon: posterior(fit, time = {horizon}). |
| bayespred.dev | Error | The bayespred.dev package is needed for this survival model. |
| crr | Error | This crr model is a competing-risks model: use it with a competing-risks outcome, Surv(time, status) with a factor status, and give `horizon`. |
| crr | Error | This crr model was fitted for failcode {failcode}, but the net benefit is for cause '{label}'. |
| multi-state coxph | Error | This coxph model is a multi-state (competing-risks) model: use it with a competing-risks outcome, Surv(time, status) with a factor status, and give `horizon`. |
| multi-state coxph | Error | Could not compute the cumulative incidences from this multi-state coxph model: {original error}. Fit it with `model = TRUE` (or keep its data available). |
| multi-state coxph | Error | Could not match the cause '{cause}' to the states of the multi-state coxph model ({states}). |
| FGR | Error | This FGR model was fitted for cause '{fitted cause}', but the net benefit is for cause '{cause}'. Fit FGR with cause = '{cause}'. |
| coxph | Error | Could not compute survival from this coxph model: {original error}. Fit it with `x = TRUE` (or `model = TRUE`). |
| survreg | Error | Some strata of the validation data are not among the survreg model's strata. |
| rstpm2 | Error | rstpm2 must be attached to predict from this {class} model: run library(rstpm2) first. |

## The prior (4)

| Type | Message |
|---|---|
| Error | `{argument}` must be two non-negative numbers: the shapes (a, b) of a Beta prior. |
| Error | `prior` must be nb_improper(), nb_uniform(), nb_jeffreys() or nb_beta(). |
| Error | For survival and competing-risks outcomes the prior is nb_improper() (the data only); other priors are for binary outcomes. |
| Error | Case weights can only be used with the improper prior, nb_improper(). |

## cIPCW: cipcw_control() (11)

| Type | Message |
|---|---|
| Error | There is no `refit` setting: the value-of-information functions always refit the censoring model in each draw. `refit_function` is an optional function(data, weights) returning the refitted censoring model. |
| Error | `refit_function` must be a function(data, weights) returning the refitted censoring model. |
| Error | `refit_function` needs the censoring `model` it refits; given weights cannot be refitted. |
| Error | Give either a censoring `model` or the weights, not both. |
| Error | `predict_function` must be a function(model, newdata, times). |
| Error | `predict_function` needs the censoring `model` it predicts from. |
| Error | `weight_event` and `weight_horizon` must be given together. |
| Error | `{argument}` must be positive, finite numbers without missing values. |
| Error | `weight_event` and `weight_horizon` must have the same length. |
| Error | `model` must be a fitted censoring model. For weights, use `weight_event` and `weight_horizon`. |
| Error | `min_prob` must be a single number strictly between 0 and 1. |

## cIPCW: censoring model (9)

| Type | Message |
|---|---|
| Warning | Could not verify that the censoring model was fitted to the censoring times (its outcome could not be read from its formula). It must be fitted with the censored patients as events, e.g. Surv(time, status == 0). |
| Error | The censoring model appears to have been fitted to the event, not to censoring. Refit it with only censoring as the event, e.g. Surv(time, status == 0). |
| Error | The censoring model appears to have been fitted to the cause of interest ({cause}), not to censoring. Refit it with only censoring as the event, e.g. Surv(time, status == 0). |
| Error | The censoring model appears to have been fitted to all events together, not to censoring. Refit it with only censoring as the event, e.g. Surv(time, status == 0). |
| Error | The censoring model appears to have been fitted to a competing cause ({other cause}), not to censoring. Refit it with only censoring as the event, e.g. Surv(time, status == 0). |
| Warning | Could not verify that the censoring model was fitted to the censoring times of `data` : its outcome does not match. It must be fitted with the censored patients as events, e.g. Surv(time, status == 0). |
| Warning | The censoring model was fitted to {n fitted} patients, but `data` has {n}. It should be fitted to the censoring times of `data` (the validation sample). |
| Error | Could not get survival probabilities from this {class} model ({original error}). As a cIPCW censoring model, give it with `predict_function` in cipcw_control(), or give the censoring weights instead. |
| Error | Unexpected output from riskRegression::predictRisk for this {class} model. |

## cIPCW: censoring weights (10)

| Type | Message |
|---|---|
| Error | The cIPCW weights have length {m} but `data` has {n} rows. |
| Warning | {k} cIPCW weights were truncated at 1 / min_prob = {1/min_prob}. |
| Warning | {k} cIPCW weights are above 100 (censoring probability below 0.01), a sign of near-violations of positivity. Consider truncation with `min_prob` . |
| Error | The censoring model returned invalid censoring probabilities (missing, or outside [0, 1]). |
| Warning | {k} censoring probabilities were truncated at min_prob = {min_prob}. This signals near-violations of positivity. |
| Error | {k} censoring probabilities are exactly 0, which gives an infinite weight (positivity is violated). Set `min_prob` to truncate them. |
| Warning | {k} censoring probabilities are below 0.01, a sign of near-violations of positivity. Consider truncation with `min_prob` . |
| Error | The censoring `predict_function` returned a curve of the wrong size: `surv` must be {n} rows by length(time) columns. |
| Error | The censoring `predict_function` must return one probability per row (or a curve list(time, surv)). |
| Error | The censoring `predict_function` returned {m} values for {n} rows; it must return one probability of remaining uncensored per row. |

## Value of information (5)

| Type | Message |
|---|---|
| Error | p_useful() takes one result of nb(): the probability that a model is useful does not depend on the other models. Compute it for each model and draw them together with plot(pu_1, pu_2); use p_best() to compare the models with each other. |
| Warning | The net benefit could not be estimated in some draws at thresholds {thresholds} (no treated patient followed up to the horizon); the value of information is NA there. |
| Error | `future_n` must be one or more positive whole numbers: the sizes of the future validation sample. |
| Warning | The net benefit could not be estimated in some draws at thresholds {thresholds} (no treated patient followed up to the horizon); EVSI is NA there. |
| Error | `n_sim` must be a whole number of at least 2. |

## cIPCW: refit in each draw (4)

| Type | Message |
|---|---|
| Error | The cIPCW censoring weights were given directly (cipcw_control(weight_event =, weight_horizon =)), so they cannot be re-estimated in each draw, and keeping them fixed would understate the uncertainty. For the value of information, give the censoring model instead: cipcw_control(model = ). |
| Error | The censoring model ({class}) must be refitted in each draw with the draw's weights (Bayesian bootstrap), and voipred does not know how to fit a {class} model with case weights. Give `refit_function = function(data, weights)` in cipcw_control(), returning the censoring model fitted with these case weights. |
| Error | Could not refit the censoring model in a draw ({original error}). Give `refit_function = function(data, weights)` in cipcw_control(), returning the censoring model fitted with these case weights. |
| Error | the model keeps no call to refit it with |

## print() and summary() (2)

| Type | Message |
|---|---|
| Error | `thresholds` must be one or more of the thresholds of the result. |
| Error | Threshold {threshold} is not among the thresholds of the result. |

## Plots (18)

| Type | Message |
|---|---|
| Error | `evpi` must be a result of evpi(). |
| Error | `evsi` must be a result of evsi(). |
| Error | The patchwork package is needed to stack the value-of-information panel under the decision curve; install it, or use `interactive = TRUE`. |
| Error | `interactive` must be TRUE or FALSE. |
| Error | The plotly package is needed for `interactive = TRUE`. |
| Error | Unknown argument `{name}`. The arguments of {function}() are {its arguments}; other arguments must be results of nb(). |
| Error | Only results of nb() can be plotted together. / {function}() needs results of nb(). |
| Error | "Treat All" and "Treat None" cannot be used as labels. |

| Type | Message |
|---|---|
| Error | The results have the same label (“{label}”); give each a different one with `labels` or nb(label = ). |
| Error | “{label 1}” and “{label 2}” cannot be used together: different {what differs}. Compute them with nb() on the same data, outcome, horizon, cause, censoring, prior and thresholds. |
| Error | Two results have the same label (“{label}”); give each a different one with `labels` . |
| Error | Unknown argument to {function}(). Its arguments are {its arguments}. |
| Error | `labels` must be one string / {n} strings, one per result. |
| Error | `thresholds` must be one or more of the thresholds of the result. |
| Error | Threshold {threshold} is not among the thresholds of the result. |
| Error | Unknown argument `{name}` . The arguments of {function}() are {its arguments}; other arguments must be results of {function}(). |
| Error | {function}() needs results of {function}(). |
| Error | Two results have the same label (“{label}”); give each a different one with `labels` . |

# Appendix: every test

| File | Test | Checks | Result |
|---|---|---|---|
| test-bayes-nb.R | with another prior, the net benefit is the posterior mean of the Beta model | 14 | pass |
| | the improper prior (default) gives exactly the frequentist net benefit | 7 | pass |
| | the improper prior handles empty groups (counts of 0) as the frequentist estimate | 8 | pass |
| | priors are checked | 17 | pass |
| test-bayes.R | one model: posterior draws equal bayesDCA's | 5 | pass |
| | EVSI equals the conjugate algorithm of Sadatsafavi et al. (2025) | 8 | pass |
| | several models are drawn jointly, each keeping its own posterior | 44 | pass |
| | multinomial draws by conditional binomials | 3 | pass |
| | results with different priors are not compared; `method` is gone | 6 | pass |
| | with the improper prior the posterior draws equal the Bayesian bootstrap in distribution | 11 | pass |
| test-cipcw.R | same as the dcurves cIPCW branch: Cox censoring model | 4 | pass |
| | same as the dcurves cIPCW branch: given weights | 2 | pass |
| | same as the dcurves cIPCW branch: competing risks, Cox censoring model | 3 | pass |
| | the vectorized estimator equals a direct implementation (ties, weights, causes) | 12 | pass |
| | a censoring model gives the same result as its weights given by hand | 4 | pass |
| | with Kaplan-Meier censoring weights, the risk by the horizon is 1 - KM | 6 | pass |
| | the censoring model is checked against the outcome | 15 | pass |
| | the censoring model's covariates must be in the data | 1 | pass |
| | small censoring probabilities: warning, optional truncation, error at 0 | 10 | pass |
| | ties: left uses the censoring survival just before each patient's time | 3 | pass |
| | every censoring model class runs and agrees with its own predictions | 28 | pass |
| | cIPCW is close to the truth under informative censoring; KM is not | 3 | pass |
| | cIPCW is close to the truth with competing risks and informative censoring | 2 | pass |
| | cIPCW with case weights equals duplicated rows | 2 | pass |
| | a censoring model with case weights equals duplicated rows and dropped rows | 7 | pass |
| | print shows the cIPCW estimator | 1 | pass |
| | censoring predict_function: same as the model route, per row or as a curve | 6 | pass |
| | censoring predict_function makes any censoring model usable | 3 | pass |
| | censoring predict_function: input and output checks | 8 | pass |
| | more censoring model classes: psm, flexsurvspline, stratified survreg | 9 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-covariates.R | glm: missing covariates are named | 2 | pass |
| | one missing covariate uses the singular | 1 | pass |
| | transformed covariates are reported by their column name | 1 | pass |
| | a model fitted with y ~ . knows all its covariates | 1 | pass |
| | ranger: covariates cannot be read generically, so ranger’s own error is shown | 2 | pass |
| | randomForest (formula and x/y interface): missing covariates are named | 2 | pass |
| | bayespred (bpmfit and bpm): missing covariates are named | 3 | pass |
| | extra columns and a missing outcome column are fine for prediction | 2 | pass |
| test-dcurves.R | logistic regression (glm) agrees with dcurves | 4 | pass |
| | random forest (ranger) agrees with dcurves, including tied risks | 4 | pass |
| | random forest (randomForest) agrees with dcurves | 5 | pass |
| | bayespred (posterior mean of risk) agrees with dcurves | 4 | pass |
| test-km-compiled.R | compiled KM / Aalen-Johansen net benefit equals the R implementation | 162 | pass |
| test-methods.R | nb() returns a list with its parts, not attributes | 14 | pass |
| | print() and summary() of nb() are short and say what the result is | 21 | pass |
| | the value-of-information results are lists with print(), summary() and as.data.frame() | 60 | pass |
| | ranges of thresholds are written compactly | 4 | pass |
| | summary() has the same columns for every outcome; without censoring Se and Sp equal the binary ones | 4 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-models.R | glm with probit and cloglog links agrees with dcurves | 8 | pass |
| | Firth logistic regression (logistf) agrees with dcurves | 4 | pass |
| | generalized additive model (mgcv::gam) agrees with dcurves | 4 | pass |
| | MARS (earth) with a binomial glm agrees with dcurves | 4 | pass |
| | classification tree (rpart) agrees with dcurves | 4 | pass |
| | conditional inference tree (partykit::ctree) agrees with dcurves | 4 | pass |
| | gradient boosting (gbm) agrees with dcurves | 4 | pass |
| | k-nearest neighbours (kknn) agrees with dcurves | 4 | pass |
| | naive Bayes (naivebayes package) agrees with dcurves | 4 | pass |
| | support vector machine (e1071::svm, probability = TRUE) agrees with dcurves | 4 | pass |
| | naive Bayes (e1071::naiveBayes) agrees with dcurves | 4 | pass |
| | neural network (nnet) agrees with dcurves, factor and 0/1 outcome | 8 | pass |
| | linear and quadratic discriminant analysis (MASS) agree with dcurves | 8 | pass |
| | svm and naiveBayes ignore missing values in columns the model does not use | 3 | pass |
| | glmnet (one lambda) and cv.glmnet agree with dcurves | 8 | pass |
| | LightGBM (objective binary) agrees with dcurves | 4 | pass |
| | XGBoost (binary:logistic) agrees with dcurves, both interfaces | 13 | pass |
| | CatBoost (Logloss) agrees with dcurves, numeric and categorical features | 8 | pass |
| | svm without probability = TRUE, or its decision values, are rejected | 2 | pass |
| | log-odds from glmnet, gbm and probit glm are rejected as scores | 2 | pass |
| | XGBoost raw log-odds (binary:logitraw) are rejected | 1 | pass |
| | glmnet with a whole lambda path is rejected | 1 | pass |
| | LightGBM raw scores and CatBoost raw values and classes are rejected | 3 | pass |
| | LDA discriminant scores are rejected | 1 | pass |
| | predicted classes from rpart and naiveBayes are rejected | 2 | pass |
| | matrix models: missing or non-numeric feature columns give clear errors | 2 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-nb.R | nb() matches a hand calculation | 5 | pass |
| | nb() returns one row per threshold with the expected columns | 6 | pass |
| | at threshold 0 the model and treat-all both equal the prevalence | 2 | pass |
| | a risk exactly at the threshold is treated (p >= z, as dcurves) | 1 | pass |
| | a risk of exactly 0 is never treated, even at threshold 0 (as dcurves) | 2 | pass |
| | unsorted thresholds are kept in the order given | 2 | pass |
| | the fast core agrees with a direct loop on random data | 1 | pass |
| | outcome can be a column name, a vector, logical, or a factor | 4 | pass |
| | weights: all ones equal no weights, integer weights equal duplicated rows | 3 | pass |
| | bad inputs give clear errors | 9 | pass |
| | the predicted risks can be given directly, without the model | 4 | pass |
| | print shows a header and the table | 2 | pass |
| | the first argument is data; the old order gives a clear error | 4 | pass |
| test-plot-voi.R | plot() of evpi(): EVPI or relative EVPI against the threshold | 10 | pass |
| | plot() of evsi(): against the future sample size | 10 | pass |
| | plot() of p_useful() and p_best(): probabilities, in the colours of the decision curve | 17 | pass |
| | plot() of nb() with an EVPI / EVSI panel below | 8 | pass |
| | interactive = TRUE gives plotly widgets with the same curves | 14 | pass |
| | static and interactive plots share one look, with the legend on the right | 12 | pass |
| | the interactive delta-NB axis title uses codes plotly can show | 3 | pass |
| | the value-of-information axis is named after what it shows | 6 | pass |
| test-plot.R | delta_NB is NB_model - max(NB_all, NB_none) | 4 | pass |
| | label: default from the model’s name, given, or reserved | 7 | pass |
| | decision curve: model, Treat All and Treat None | 8 | pass |
| | delta plot: delta_NB of each model, line at 0 | 3 | pass |
| | several results on one plot, for both types | 12 | pass |
| | results that cannot be drawn together are refused | 11 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-predict_risk.R | glm: predict_risk() equals predict(type = ‘response’) | 1 | pass |
| | a glm whose predictions are not probabilities is rejected | 1 | pass |
| | ranger: the risk is the probability of class 1 | 1 | pass |
| | ranger: numeric 0/1 outcome (unnamed columns) gives the second column | 1 | pass |
| | ranger: a class-only forest is rejected with advice | 1 | pass |
| | randomForest: the risk is the probability of class 1 | 1 | pass |
| | a forest with more than two classes is rejected | 1 | pass |
| | bayespred: the risk is the posterior mean of risk | 3 | pass |
| | a numeric or logical vector is rejected | 2 | pass |
| | an object without a usable predict() gets a clear error | 1 | pass |
| | default method works for a model with predict(type = ‘response’) | 1 | pass |
| | missing covariates are caught | 1 | pass |
| test-prediction-routes.R | predict_function takes precedence over voipred’s own method and the standard route | 3 | pass |
| | predict_function gets horizon and cause; it is checked like any model | 12 | pass |
| | output shapes are read generically | 10 | pass |
| | standard binary route: type = ‘response’, then ‘prob’, then riskRegression | 2 | pass |
| | standard survival route: riskRegression::predictRisk, cause by label or position | 2 | pass |
| | a model with no usable route gets one clear error with the ways out | 3 | pass |
| | a new survival model works with predict_function | 3 | pass |
| | predict_risk() is internal | 2 | pass |
| test-probabilities.R | only 0s and 1s are rejected as classes, not probabilities | 3 | pass |
| | integer 0/1 predictions are rejected too | 1 | pass |
| | probabilities that include some exact 0s and 1s are accepted | 2 | pass |
| | a random forest regression on a 0/1 outcome still works | 2 | pass |
| | a glm asked for log-odds would be outside [0, 1]: predict_risk uses probabilities | 2 | pass |
| | values above 1 or below 0 are rejected as not probabilities | 3 | pass |
| | log-odds from a glm would be rejected as scores | 1 | pass |
| | a linear probability model (lm) with predictions below 0 is rejected | 2 | pass |
| test-risk-input.R | binary: risks given directly give the same results as the model | 11 | pass |
| | survival and competing risks, with Kaplan-Meier and cIPCW | 4 | pass |
| | a model’s risks are predicted once and reused by every later step | 2 | pass |
| | `predicted_risk` is checked | 10 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-survival-competing.R | riskRegression::CSC for both causes, with numeric or labelled causes (standard route) | 21 | pass |
| | riskRegression::FGR (standard route) and cmprsk::crr (voipred method) agree | 18 | pass |
| | randomForestSRC competing-risks forest (standard route, cause by position) | 14 | pass |
| | survival's multi-state coxph: Aalen-Johansen from cause-specific Cox models | 17 | pass |
| | multi-state coxph: causes matched by label or position, and the checks | 5 | pass |
| | a Fine-Gray model for another cause is rejected (riskRegression would not notice) | 3 | pass |
| | a standard survival model with a competing outcome works, with a warning | 4 | pass |
| | a competing-risks model needs a competing-risks outcome | 10 | pass |
| | missing covariates are reported for competing-risks models | 4 | pass |
| | numeric competing-risks codes get a hint to use a factor status | 1 | pass |
| test-survival-km.R | KM net benefit equals dcurves at every threshold, including NA thresholds | 71 | pass |
| | the risk by the horizon (treat all) is 1 - Kaplan-Meier | 1 | pass |
| | the internal KM and Aalen-Johansen match survival::survfit, with weights | 9 | pass |
| | no follow-up to the horizon among the treated gives NA and a warning | 3 | pass |
| | nobody treated gives NB_model = 0 | 1 | pass |
| | a risk at the threshold is treated and a risk of 0 never is | 2 | pass |
| | weights: all ones equal no weights; integer weights equal duplicated rows | 3 | pass |
| | Aalen-Johansen net benefit equals dcurves for both causes | 7 | pass |
| | with competing risks, weights equal duplicated rows | 1 | pass |
| | KM is unbiased with independent censoring and biased with informative censoring | 2 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-survival-models.R | coxph fitted with x = TRUE (Efron, Breslow, case weights, ties, strata, splines) | 30 | pass |
| | rms::cph and rms::psm | 12 | pass |
| | survreg (Weibull, log-normal, log-logistic, exponential, strata) | 32 | pass |
| | flexsurvreg and flexsurvspline (formula written in the call or stored in a variable) | 19 | pass |
| | random survival forests: ranger and randomForestSRC | 12 | pass |
| | prodlim | 6 | pass |
| | coxph without x = TRUE (riskRegression refuses it): own baseline hazard | 32 | pass |
| | a coxph model that needs its (unavailable) data gives a clear error | 1 | pass |
| | the own Cox baseline hazard equals survival::basehaz | 2 | pass |
| | aorsf oblique survival forest | 7 | pass |
| | partykit::ctree survival tree | 7 | pass |
| | rstpm2::stpm2 flexible parametric model (needs rstpm2 attached) | 7 | pass |
| | mgcv::gam with the Cox PH family | 9 | pass |
| | models giving only relative risks or times have no route: clear error | 6 | pass |
| | a binary model with a survival outcome works, with a warning | 5 | pass |
| | survival models with a binary outcome are rejected | 9 | pass |
| | a horizon beyond the model’s own follow-up gives a warning | 6 | pass |
| | missing covariates are reported for survival models | 11 | pass |
| | each model’s risk grows with the horizon | 2 | pass |
| test-survival-outcome.R | the survival outcome can be given as a formula, a Surv object or a column | 5 | pass |
| | a binary outcome can also be given as a formula | 1 | pass |
| | horizon is required and must be a positive number within follow-up | 6 | pass |
| | survival-only arguments are rejected for a binary outcome | 4 | pass |
| | cause is only for competing risks and must be an event level | 2 | pass |
| | only right-censored outcomes are supported | 2 | pass |
| | missing or negative times and missing statuses are rejected | 5 | pass |
| | censoring settings must be consistent | 4 | pass |
| | cipcw_control() checks its inputs | 11 | pass |
| | cIPCW weights must have one value per row | 1 | pass |
| | print shows the outcome type, horizon and estimator | 2 | pass |

| File | Test | Checks | Result |
|---|---|---|---|
| test-voi-refit.R | every known censoring model is refitted with the draw’s weights | 24 | pass |
| | other censoring models need refit_function | 5 | pass |
| | refit_function replaces the automatic refit | 3 | pass |
| | models validated with the same censoring settings share one refit per draw | 1 | pass |
| | EVPI, P(useful) and EVSI with the refit, also for competing risks | 6 | pass |
| | given censoring weights work in nb() but not in the value of information | 5 | pass |
| | refit settings are checked | 5 | pass |
| test-voi.R | draws with all weights 1 reproduce nb() for every outcome and estimator | 12 | pass |
| | EVPI and P(useful) equal predtools::evpi_val() (Sadatsafavi et al. 2023) | 17 | pass |
| | EVSI equals the bootstrap algorithm of Sadatsafavi et al. (2025) | 5 | pass |
| | EVSI is between 0 and EVPI, grows with the future sample, and reaches EVPI | 9 | pass |
| | P(useful), P(best) and relative EVPI follow their definitions | 14 | pass |
| | relative EVPI is Inf when no model is better than the defaults but EVPI > 0 | 2 | pass |
| | a survival outcome without censoring before the horizon gives the binary results | 4 | pass |
| | survival, competing risks and cIPCW: EVPI, EVSI, P(useful), P(best) run and are coherent | 6 | pass |
| | thresholds where the net benefit cannot be estimated give NA, with a warning | 2 | pass |
| | case weights are part of the draws | 1 | pass |
| | inputs are checked | 15 | pass |